

Instrucciones para IA de Codificación — Aplicación Web Dashboard Metro Chile

1. Rol que debe asumir la IA de desarrollo

Asume el rol de un desarrollador de software senior, arquitecto de software e ingeniero de software experto en aplicaciones web, APIs, sistemas modulares, integración de dispositivos IoT/visión computacional y dashboards operacionales para clientes industriales.

Debes desarrollar una aplicación web modular, mantenible y preparada para integración con dispositivos externos desarrollados por otro equipo. El objetivo principal es construir un dashboard web que reciba, procese, almacene, visualice y exporte datos provenientes de dos tipos de dispositivos de visión computacional utilizados en un proyecto piloto para Metro de Chile.

La solución debe estar diseñada con buenas prácticas de ingeniería, separación clara de responsabilidades, documentación interna y facilidad de integración futura.

2. Contexto del proyecto

El cliente es la empresa Metro de Chile.

El sistema forma parte de un piloto tecnológico orientado a seguridad operacional y prevención de riesgos en frentes de trabajo. Los dispositivos de visión computacional serán desarrollados por otros miembros del equipo, por lo tanto, esta aplicación web debe estar preparada para recibir los datos cuando dichos dispositivos estén listos.

La aplicación web no controla directamente los dispositivos. Su responsabilidad es:

- Recibir datos enviados por los dispositivos o por el gateway/intermediario.
 - Validar los JSON entrantes.
 - Normalizar la información.
 - Almacenar datos históricos.
 - Mostrar métricas en un dashboard.
 - Permitir descarga manual de la información en formato tabular.
 - Aislar todos los parámetros de integración para facilitar cambios futuros.
-

3. Tecnologías obligatorias

Frontend

Usar:

- HTML
- CSS

- JavaScript nativo

No usar frameworks frontend como React, Angular o Vue, salvo que se indique explícitamente en una etapa posterior.

Backend

Usar:

- Python
- FastAPI como framework recomendado
- Pydantic para validación de datos
- SQLAlchemy para modelos y persistencia
- SQLite para desarrollo local
- PostgreSQL como opción preparada para despliegue cloud

Exportación

La aplicación debe permitir exportar datos manualmente en:

- CSV como mínimo obligatorio
- XLSX como opción deseable

4. Descripción funcional general

La aplicación debe incluir:

1. Login de usuario.
2. Dashboard principal.
3. Panel de detecciones EPP.
4. Panel de detecciones del frente de trabajo.
5. Tabla histórica de eventos.
6. Filtros por fecha, dispositivo y tipo de evento.
7. Descarga manual de datos.
8. Backend con endpoints para recepción de datos.
9. Backend con endpoints para consulta de métricas.
10. Backend con validación y normalización de JSON.
11. Almacenamiento del JSON original recibido.
12. Almacenamiento del dato procesado/normalizado.
13. Carpeta centralizada de configuración para modificar parámetros de integración sin tocar la lógica principal.

5. Usuarios del sistema

Inicialmente el sistema debe estar preparado para aproximadamente 5 usuarios.

A futuro podría escalar hasta un máximo cercano a 10 usuarios durante el piloto.

No es necesario diseñar una arquitectura masiva de usuarios, pero sí se debe implementar autenticación correctamente.

Roles mínimos sugeridos:

- `admin`: puede ver dashboard, gestionar usuarios y exportar datos.
- `viewer`: puede ver dashboard y descargar datos.

Si se desea simplificar en primera versión, implementar solo login con usuarios predefinidos y contraseña hasheada, pero dejando preparada la estructura para roles.

6. Dispositivo 1: Detección de EPP

Propósito operacional

El dispositivo de detección de EPP tiene como finalidad cuantificar el correcto uso de equipos de protección personal por parte de los trabajadores.

Este dispositivo puede detectar a varios trabajadores a la vez. Además, una detección puede persistir durante un tiempo, por lo que no se debe asumir que cada JSON representa necesariamente un trabajador único o un evento instantáneo aislado.

Los EPP detectados son:

- Casco
- Guantes
- Gafas
- Chaleco reflectante
- Mascarilla

Valor para prevencionistas de riesgos

La información debe permitir analizar:

- Cumplimiento general de uso de EPP.
- Cantidad de trabajadores detectados.
- Cantidad de trabajadores con EPP completo.
- Cantidad de trabajadores con EPP incompleto.
- EPP faltantes más frecuentes.
- Tendencias por fecha, hora y ubicación.
- Eventos críticos de incumplimiento.

Valor para análisis de datos del cliente

La estructura debe permitir:

- Análisis temporal.
- Agrupación por dispositivo.
- Agrupación por ubicación.
- Tasa de cumplimiento.

- Identificación de patrones horarios.
- Exportación histórica para Power BI, Excel u otra herramienta analítica.

7. JSON recomendado para dispositivo EPP

Este formato debe ser tratado como contrato inicial de integración. Debe estar centralizado y documentado para que pueda modificarse fácilmente cuando el equipo de dispositivos entregue el formato definitivo.

```
{
  "event_type": "EPP_DETECTION",
  "schema_version": "1.0",
  "event_id": "evt-epp-000001",
  "device_id": "EPP-001",
  "device_type": "EPP_CAMERA",
  "client": "Metro Chile",
  "location": {
    "site": "Linea 7",
    "area": "Pique de trabajo",
    "zone": "Acceso principal",
    "camera_position": "Ingreso frente de trabajo"
  },
  "timestamp": "2026-05-05T10:30:00-04:00",
  "detection_window": {
    "start_time": "2026-05-05T10:29:55-04:00",
    "end_time": "2026-05-05T10:30:00-04:00",
    "duration_seconds": 5
  },
  "summary": {
    "workers_detected": 3,
    "workers_full_compliance": 1,
    "workers_partial_compliance": 2,
    "workers_without_required_ppe": 0,
    "overall_compliance_percentage": 73.33
  },
  "workers": [
    {
      "temporary_tracking_id": "track-001",
      "confidence": 0.94,
      "ppe": {
        "helmet": {
          "detected": true,
          "confidence": 0.98
        },
        "gloves": {
          "detected": false,
          "confidence": 0.76
        },
        "goggles": {
```

```

        "detected": true,
        "confidence": 0.91
    },
    "reflective_vest": {
        "detected": true,
        "confidence": 0.96
    },
    "mask": {
        "detected": false,
        "confidence": 0.65
    }
},
"compliance_status": "PARTIAL",
"missing_ppe": [
    "gloves",
    "mask"
]
},
{
    "temporary_tracking_id": "track-002",
    "confidence": 0.90,
    "ppe": {
        "helmet": {
            "detected": true,
            "confidence": 0.97
        },
        "gloves": {
            "detected": true,
            "confidence": 0.88
        },
        "goggles": {
            "detected": true,
            "confidence": 0.89
        },
        "reflective_vest": {
            "detected": true,
            "confidence": 0.95
        },
        "mask": {
            "detected": true,
            "confidence": 0.84
        }
    },
    "compliance_status": "FULL",
    "missing_ppe": []
}
],
"alerts": {
    "non_compliance_detected": true,
    "critical_missing_ppe": false,
    "alert_level": "MEDIUM"
}

```

```
},  
"raw_metadata": {  
  "model_name": "pending_device_model",  
  "model_version": "pending",  
  "gateway_id": "LORA-GW-001",  
  "transmission_type": "LoRaWAN"  
}  
}
```

Consideraciones importantes para EPP

1. No asumir que siempre habrá un solo trabajador.
2. No asumir que el evento es instantáneo; debe existir ventana de detección.
3. Guardar porcentajes y conteos resumidos para dashboard.
4. Guardar detalle por trabajador detectado para análisis posterior.
5. Usar identificadores temporales, no nombres personales.
6. Evitar almacenar datos biométricos o personales.
7. Guardar siempre el JSON original en base de datos.
8. Normalizar los nombres de EPP internamente en inglés técnico o español consistente, pero no mezclar ambos sin control.

8. Dispositivo 2: Frente de trabajo / excavación

Propósito operacional

El dispositivo ubicado en el frente de trabajo tiene como finalidad detectar piedras de gran tamaño y deslizamientos de tierra que puedan representar una condición de riesgo para los trabajadores o la operación.

El sistema debe poder registrar detecciones y posteriormente reflejar si se activó o no una alarma asociada a la detección.

Eventos detectados

- Piedras de gran tamaño.
- Deslizamientos de tierra.
- Activación de alarma.

Valor para prevencionistas de riesgos

La información debe permitir analizar:

- Cantidad de eventos de riesgo detectados.
- Severidad de cada evento.
- Hora y ubicación de detecciones.
- Frecuencia de detecciones.
- Eventos con activación de alarma.
- Eventos sin activación de alarma.
- Tiempo entre detección y activación de alarma.

- Zonas con mayor recurrencia de riesgo.

Valor para análisis de datos del cliente

La estructura debe permitir:

- Series temporales de eventos.
- Análisis por ubicación y dispositivo.
- Clasificación por severidad.
- Análisis de recurrencia.
- Comparación entre detecciones y activaciones de alarma.
- Integración futura con Power BI o sistemas de reporting.

9. JSON recomendado para dispositivo de excavación

```
{
  "event_type": "EXCAVATION_RISK_DETECTION",
  "schema_version": "1.0",
  "event_id": "evt-exc-000001",
  "device_id": "EXC-001",
  "device_type": "EXCAVATION_CAMERA",
  "client": "Metro Chile",
  "location": {
    "site": "Linea 7",
    "area": "Pique de trabajo",
    "zone": "Frente de excavacion",
    "camera_position": "Frente de trabajo principal"
  },
  "timestamp": "2026-05-05T10:35:00-04:00",
  "detection_window": {
    "start_time": "2026-05-05T10:34:58-04:00",
    "end_time": "2026-05-05T10:35:00-04:00",
    "duration_seconds": 2
  },
  "risk_detection": {
    "large_rocks_detected": true,
    "large_rocks_count": 2,
    "landslide_detected": false,
    "landslide_count": 0,
    "movement_detected": true,
    "risk_level": "HIGH",
    "confidence": 0.92
  },
  "detected_objects": [
    {
      "object_type": "LARGE_ROCK",
      "object_id": "rock-001",
      "confidence": 0.93,
      "estimated_size_category": "LARGE",
    }
  ]
}
```

```

    "position_reference": {
      "x_center": 0.45,
      "y_center": 0.62,
      "width": 0.18,
      "height": 0.22
    }
  },
  {
    "object_type": "LARGE_ROCK",
    "object_id": "rock-002",
    "confidence": 0.89,
    "estimated_size_category": "MEDIUM",
    "position_reference": {
      "x_center": 0.66,
      "y_center": 0.58,
      "width": 0.15,
      "height": 0.19
    }
  }
],
"alarm": {
  "alarm_triggered": true,
  "alarm_type": "VISUAL_BEACON",
  "alarm_level": "HIGH",
  "trigger_reason": "Large rocks detected in excavation front",
  "triggered_at": "2026-05-05T10:35:01-04:00",
  "activation_duration_seconds": 10
},
"raw_metadata": {
  "model_name": "pending_device_model",
  "model_version": "pending",
  "gateway_id": "LORA-GW-001",
  "transmission_type": "LoRaWAN"
}
}

```

Consideraciones importantes para excavación

1. Separar detección de piedras y detección de deslizamiento.
2. Registrar severidad o nivel de riesgo.
3. Registrar si se activó alarma.
4. Registrar motivo de activación de alarma.
5. Registrar duración de activación, si el dato está disponible.
6. Guardar detalle de objetos detectados si el dispositivo puede entregarlo.
7. No hacer que el dashboard dependa obligatoriamente del detalle de bounding boxes, porque puede que el dispositivo final no entregue esos datos.
8. Guardar siempre el JSON original.

10. Principio clave de arquitectura: desacoplamiento de integración

Debido a que los dispositivos son desarrollados por otro equipo, la aplicación debe aislar todas las variables de integración en una carpeta específica.

Debe existir una carpeta llamada:

```
backend/app/integration_config/
```

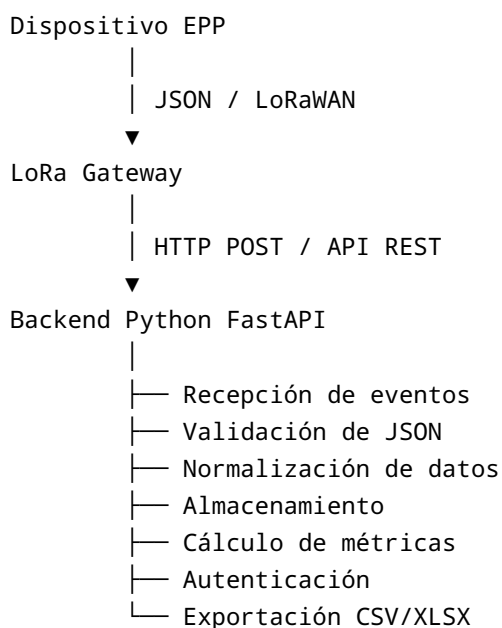
Esta carpeta debe contener todos los archivos relacionados con:

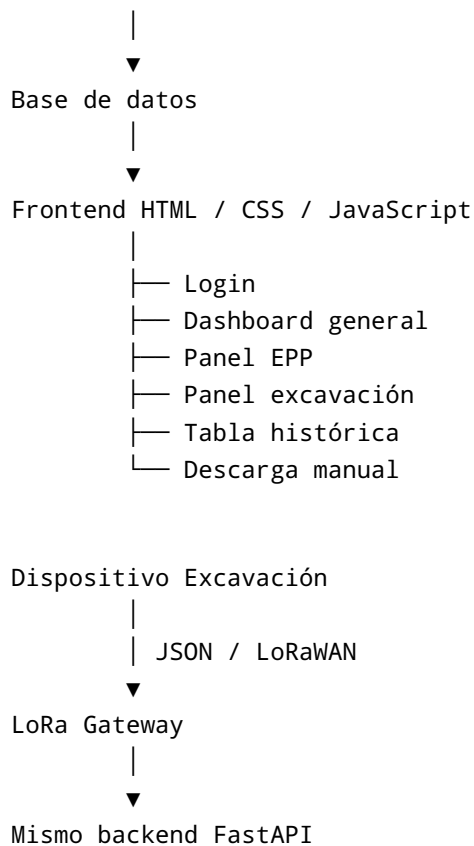
- Nombres de campos esperados.
- Versiones de esquemas.
- Mapeo de campos externos a campos internos.
- IDs de dispositivos autorizados.
- Tipos de dispositivos.
- Parámetros de validación.
- Umbrales de severidad.
- Configuración de gateway.
- Configuración de endpoints externos, si aplica.

El objetivo es que, cuando el equipo de dispositivos entregue el JSON final, solo sea necesario modificar archivos dentro de esta carpeta y, como máximo, los schemas/normalizers asociados.

No se debe dispersar la lógica de integración por todo el backend.

11. Arquitectura general de la solución





12. Estructura de carpetas obligatoria

La aplicación debe organizarse de la siguiente forma:

```
metro-safety-dashboard/  
├── backend/  
│   ├── app/  
│   │   ├── main.py  
│   │   ├── config.py  
│   │   └── dependencies.py  
│   ├── integration_config/  
│   │   ├── device_types.py  
│   │   ├── field_mappings.py  
│   │   ├── schema_versions.py  
│   │   ├── thresholds.py  
│   │   ├── authorized_devices.py  
│   │   └── gateway_config.py  
│   ├── auth/  
│   │   ├── routes.py  
│   │   └── services.py
```

```

├── security.py
├── models.py
├── api/
│   ├── epp_routes.py
│   ├── excavation_routes.py
│   ├── dashboard_routes.py
│   ├── export_routes.py
│   └── health_routes.py
├── schemas/
│   ├── epp_schema.py
│   ├── excavation_schema.py
│   ├── normalized_event_schema.py
│   ├── dashboard_schema.py
│   └── user_schema.py
├── services/
│   ├── epp_service.py
│   ├── excavation_service.py
│   ├── dashboard_service.py
│   ├── export_service.py
│   └── device_service.py
├── normalizers/
│   ├── epp_normalizer.py
│   ├── excavation_normalizer.py
│   └── common_normalizer.py
├── validators/
│   ├── epp_validator.py
│   ├── excavation_validator.py
│   └── common_validator.py
├── models/
│   ├── user.py
│   ├── device.py
│   ├── raw_event.py
│   ├── epp_event.py
│   └── excavation_event.py
├── database/
│   ├── connection.py
│   ├── session.py
│   ├── init_db.py
│   └── migrations/
├── repositories/
│   ├── user_repository.py
│   └── event_repository.py

```

```

├── epp_repository.py
├── excavation_repository.py
├── utils/
│   ├── date_utils.py
│   ├── response_utils.py
│   ├── csv_utils.py
│   └── xlsx_utils.py
├── tests/
│   ├── test_epp_ingestion.py
│   ├── test_excavation_ingestion.py
│   ├── test_auth.py
│   └── test_export.py
├── requirements.txt
├── .env.example
├── README.md
├── frontend/
│   ├── index.html
│   ├── login.html
│   ├── dashboard.html
│   ├── events.html
│   ├── css/
│   │   ├── base.css
│   │   ├── layout.css
│   │   ├── login.css
│   │   ├── dashboard.css
│   │   ├── tables.css
│   │   └── components.css
│   ├── js/
│   │   ├── config.js
│   │   ├── api.js
│   │   ├── auth.js
│   │   ├── dashboard.js
│   │   ├── epp.js
│   │   ├── excavation.js
│   │   ├── events_table.js
│   │   ├── export.js
│   │   └── utils.js
│   └── assets/
│       ├── logo_metro_placeholder.png
│       └── icons/
├── docs/
│   └── architecture.md

```

```
| | | | api_contract.md
| | | | json_examples.md
| | | | integration_notes.md
| | | | deployment_notes.md
| | |
| | | docker-compose.yml
| | | README.md
| | | .gitignore
```

13. Descripción de carpetas y archivos

`backend/app/main.py`

Archivo principal de FastAPI. Debe inicializar la aplicación, registrar routers, habilitar CORS y cargar configuración general.

`backend/app/config.py`

Debe centralizar variables generales del backend:

- Ambiente de ejecución.
- URL de base de datos.
- Secret key JWT.
- Duración de tokens.
- Configuración de CORS.

`backend/app/integration_config/`

Carpeta crítica del proyecto. Debe contener todos los parámetros modificables para integración con dispositivos externos.

`device_types.py`

Define tipos de dispositivos aceptados:

- `EPP_CAMERA`
- `EXCAVATION_CAMERA`

`field_mappings.py`

Define cómo se traducen campos externos del JSON a campos internos del sistema.

Ejemplo:

```
EPP_FIELD_MAPPING = {
    "helmet": "casco",
    "gloves": "guantes",
    "goggles": "gafas",
```

```
"reflective_vest": "chaleco_reflectante",  
"mask": "mascarilla"  
}
```

schema_versions.py

Define versiones soportadas de esquema:

```
SUPPORTED_SCHEMA_VERSIONS = ["1.0"]
```

thresholds.py

Define umbrales configurables:

- Confianza mínima de detección.
- Porcentaje mínimo aceptable de cumplimiento EPP.
- Nivel de riesgo para alarma.
- Cantidad mínima de rocas para riesgo alto.

authorized_devices.py

Define dispositivos autorizados para enviar datos:

```
AUTHORIZED_DEVICES = {  
    "EPP-001": "EPP_CAMERA",  
    "EXC-001": "EXCAVATION_CAMERA"  
}
```

gateway_config.py

Define parámetros relacionados con el gateway:

- ID esperado del gateway.
- Tipo de transmisión.
- Token/API key, si corresponde.

backend/app/auth/

Contiene todo lo relacionado con autenticación:

- Login.
 - Generación de JWT.
 - Validación de usuario.
 - Hash de contraseñas.
 - Roles.
-

backend/app/api/

Contiene las rutas HTTP de la API.

Debe incluir como mínimo:

```
POST /api/epp/events
POST /api/excavation/events
GET /api/dashboard/summary
GET /api/dashboard/epp
GET /api/dashboard/excavation
GET /api/events
GET /api/export/csv
GET /api/export/xlsx
GET /api/health
```

backend/app/schemas/

Contiene modelos Pydantic para validar entrada y salida de datos.

Debe haber schemas independientes para:

- JSON EPP.
- JSON excavación.
- Evento normalizado.
- Usuario.
- Respuestas de dashboard.

backend/app/normalizers/

Contiene la lógica que transforma el JSON recibido en una estructura interna estable.

Este es uno de los módulos más importantes del proyecto.

Regla obligatoria:

El frontend y los servicios de dashboard nunca deben depender directamente del JSON crudo enviado por los dispositivos.

El flujo correcto es:

```
JSON recibido → Validación → Normalización → Modelo interno → Base de datos → Dashboard
```

`backend/app/validators/`

Contiene validaciones adicionales a las de Pydantic:

- Dispositivo autorizado.
 - Versión de esquema soportada.
 - Campos mínimos requeridos.
 - Coherencia de fechas.
 - Coherencia de conteos.
 - Valores de confianza entre 0 y 1.
-

`backend/app/models/`

Contiene los modelos SQLAlchemy para base de datos.

Modelos mínimos:

- Usuario.
 - Dispositivo.
 - Evento crudo.
 - Evento EPP procesado.
 - Evento excavación procesado.
-

`backend/app/repositories/`

Contiene acceso a base de datos.

La lógica de negocio no debe consultar directamente SQLAlchemy desde los routers. Debe usar repositorios.

`backend/app/services/`

Contiene lógica de negocio.

Ejemplo:

- Calcular cumplimiento EPP.
 - Calcular resumen de riesgo.
 - Preparar datos para dashboard.
 - Preparar exportaciones.
-

`frontend/js/config.js`

Debe centralizar la URL base del backend y variables modificables del frontend.

Ejemplo:

```
const API_BASE_URL = "http://localhost:8000";  
const DASHBOARD_REFRESH_INTERVAL_MS = 5000;
```

frontend/js/api.js

Debe contener funciones reutilizables para consumir el backend.

Ejemplo:

- getDashboardSummary()
- getEppEvents()
- getExcavationEvents()
- downloadCsv()

frontend/js/dashboard.js

Debe controlar la actualización del dashboard.

Para el piloto, utilizar polling cada 5 segundos.

No implementar WebSockets en primera versión salvo que sea estrictamente necesario.

14. Base de datos recomendada

Para desarrollo local:

SQLite

Para despliegue cloud:

PostgreSQL

El código debe estar preparado para cambiar la base de datos mediante variable de entorno.

15. Tablas mínimas

Tabla `users`

```
id
username
email
password_hash
role
is_active
created_at
updated_at
```

Tabla `devices`

```
id
device_id
device_type
client
site
area
zone
description
is_active
created_at
updated_at
```

Tabla `raw_events`

```
id
event_id
event_type
schema_version
device_id
received_at
raw_payload
validation_status
validation_errors
```

Tabla `epp_events`

```
id
raw_event_id
event_id
device_id
timestamp
site
```

```
area
zone
workers_detected
workers_full_compliance
workers_partial_compliance
workers_without_required_ppe
overall_compliance_percentage
missing_helmet_count
missing_gloves_count
missing_goggles_count
missing_reflective_vest_count
missing_mask_count
alert_level
non_compliance_detected
created_at
```

Table excavation_events

```
id
raw_event_id
event_id
device_id
timestamp
site
area
zone
large_rocks_detected
large_rocks_count
landslide_detected
landslide_count
movement_detected
risk_level
confidence
alarm_triggered
alarm_type
alarm_level
trigger_reason
activation_duration_seconds
created_at
```

16. Endpoints obligatorios

Autenticación

```
POST /auth/login
GET /auth/me
POST /auth/logout
```

Ingesta de datos EPP

```
POST /api/epp/events
```

Debe:

1. Recibir JSON.
2. Guardar JSON crudo.
3. Validar estructura.
4. Validar dispositivo autorizado.
5. Normalizar datos.
6. Guardar evento procesado.
7. Retornar estado de recepción.

Ingesta de datos excavación

```
POST /api/excavation/events
```

Debe realizar el mismo flujo anterior, adaptado a excavación.

Dashboard

```
GET /api/dashboard/summary
GET /api/dashboard/epp
GET /api/dashboard/excavation
```

Debe retornar métricas listas para frontend.

Eventos históricos

```
GET /api/events
```

Debe permitir filtros:

```
?event_type=
?device_id=
```

```
?start_date=  
?end_date=  
?limit=  
?offset=
```

Exportación

```
GET /api/export/csv  
GET /api/export/xlsx
```

Debe permitir los mismos filtros que `/api/events`.

Health check

```
GET /api/health
```

Debe retornar estado del backend.

17. Dashboard frontend

El dashboard debe mostrar como mínimo:

Resumen general

- Total de eventos recibidos.
- Última actualización.
- Estado de conexión con backend.
- Cantidad de dispositivos activos.
- Eventos críticos recientes.

Panel EPP

- Trabajadores detectados.
- Porcentaje de cumplimiento general.
- Cantidad de trabajadores con EPP completo.
- Cantidad de trabajadores con EPP parcial.
- EPP faltante más frecuente.
- Últimas detecciones de incumplimiento.

Panel excavación

- Total de detecciones de rocas grandes.
- Total de detecciones de deslizamiento.
- Nivel de riesgo actual.
- Última alarma activada.
- Cantidad de alarmas activadas.
- Últimos eventos críticos.

Tabla histórica

- Fecha/hora.
 - Tipo de evento.
 - ID dispositivo.
 - Ubicación.
 - Resumen de evento.
 - Nivel de alerta/riesgo.
 - Botón o acción para ver detalle.
-

18. Actualización en tiempo real

Para la primera versión del piloto, usar polling desde el frontend.

Configurar en:

```
frontend/js/config.js
```

Ejemplo:

```
const DASHBOARD_REFRESH_INTERVAL_MS = 5000;
```

El dashboard debe consultar periódicamente:

```
/api/dashboard/summary  
/api/dashboard/epp  
/api/dashboard/excavation
```

No implementar WebSockets inicialmente, a menos que se indique expresamente.

19. Reglas de diseño visual

El frontend debe tener una apariencia:

- Profesional.
- Sobria.
- Clara.
- Orientada a operación y seguridad.
- Compatible con pantallas de escritorio.

Usar tarjetas de indicadores, tablas claras y colores semánticos:

- Verde: estado normal/cumplimiento.
- Amarillo: advertencia.

- Rojo: riesgo/incumplimiento crítico.
- Gris/azul: información general.

No sobrecargar visualmente el dashboard.

20. Reglas críticas de modularidad

1. No escribir nombres de campos del JSON directamente en múltiples partes del código.
 2. Centralizar mappings en `integration_config/field_mappings.py`.
 3. Centralizar umbrales en `integration_config/thresholds.py`.
 4. Centralizar IDs de dispositivos en `integration_config/authorized_devices.py`.
 5. Guardar siempre el JSON original.
 6. El dashboard debe consumir datos normalizados, no JSON crudo.
 7. Los endpoints de ingesta deben estar separados por tipo de dispositivo.
 8. El código debe incluir comentarios donde se espera modificación futura por integración.
 9. Deben existir ejemplos de JSON en `docs/json_examples.md`.
 10. Deben existir notas de integración en `docs/integration_notes.md`.
-

21. Seguridad mínima requerida

Implementar:

- Login con contraseña hasheada.
- JWT o sesión segura.
- Protección de rutas del dashboard.
- Protección de endpoints de consulta.
- API key opcional para endpoints que reciben datos desde gateway.
- CORS configurado correctamente.
- Variables sensibles en `.env`.

No dejar credenciales duras en el código.

22. Requerimientos de documentación

Crear documentación mínima en:

```
docs/architecture.md
docs/api_contract.md
docs/json_examples.md
docs/integration_notes.md
docs/deployment_notes.md
```

La documentación debe explicar:

- Arquitectura general.

- Flujo de datos.
 - Endpoints disponibles.
 - Estructura JSON esperada.
 - Cómo cambiar mappings cuando los dispositivos entreguen el formato final.
 - Cómo ejecutar localmente.
 - Cómo desplegar en cloud.
-

23. Requerimientos de pruebas

Crear pruebas básicas para:

- Recepción de JSON EPP válido.
 - Rechazo de JSON EPP inválido.
 - Recepción de JSON excavación válido.
 - Rechazo de JSON excavación inválido.
 - Login correcto.
 - Exportación CSV.
 - Validación de dispositivo autorizado.
-

24. Resultado esperado

La IA de codificación debe entregar una aplicación funcional de primera versión con:

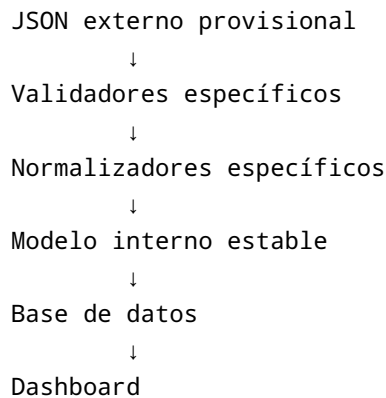
1. Backend FastAPI ejecutable.
 2. Frontend HTML/CSS/JS funcional.
 3. Login básico.
 4. Endpoints de recepción para ambos dispositivos.
 5. Validación de JSON.
 6. Normalización de datos.
 7. Base de datos local.
 8. Dashboard visual.
 9. Tabla histórica.
 10. Exportación CSV.
 11. Estructura modular completa.
 12. Archivos de configuración de integración centralizados.
 13. Documentación mínima.
 14. Ejemplos de JSON.
 15. Código comentado en puntos de integración futura.
-

25. Advertencia técnica importante

No desarrollar la aplicación como un prototipo rígido acoplado a los JSON actuales.

Los JSON propuestos son contratos iniciales, pero el equipo de dispositivos puede modificar el formato final. Por eso, la aplicación debe estar preparada para adaptación rápida.

El diseño correcto es:



Nunca conectar directamente el frontend al JSON crudo.

26. Prioridad de desarrollo

Orden recomendado:

1. Crear estructura de carpetas.
 2. Configurar backend FastAPI.
 3. Crear modelos de base de datos.
 4. Crear schemas Pydantic.
 5. Crear integration_config.
 6. Crear endpoints de ingesta.
 7. Crear normalizadores.
 8. Crear almacenamiento de raw events.
 9. Crear endpoints de dashboard.
 10. Crear login.
 11. Crear frontend.
 12. Crear tabla histórica.
 13. Crear exportación CSV.
 14. Crear documentación.
 15. Crear pruebas básicas.
-

27. Criterio final de aceptación

La aplicación se considerará aceptable si:

- Puede ejecutarse localmente.
- Permite iniciar sesión.
- Recibe JSON EPP de ejemplo.
- Recibe JSON excavación de ejemplo.
- Guarda los eventos.
- Muestra datos en dashboard.

- Permite ver historial.
- Permite descargar CSV.
- El código es modular.
- Los parámetros de integración están centralizados.
- Se puede modificar el formato esperado del JSON sin rehacer toda la aplicación.